Mo Tu We Th Fr Sa Su

Memo No. _____

Date / /

CS 336

第二节 PyTorch

Scaling law (缩放定律): It's an empirical law in LLM area. It describes how model performance chances when as you increase resources as such as:

Model size N: number of parameters

training data D: number of training tokens

compute C: ~~traing~~ training FLOPs

A typical scaling law says that as you spend more compute, use a large model, or training on more data, the losses tends to decrease in a fairly predictable way.

A simplified form is:

$$\underbrace{L(N, D)}_{\text{model loss}} = \underbrace{L_\infty}_{\text{the irreducible lower bound of Loss}} + \frac{A}{N^\alpha} + \frac{B}{D^\beta}$$

A, B, α, β: constants fitted (拟合) from ~~expre~~ experiments

Mo Tu We Th Fr Sa Su

Memo No. _____

Date the system/

Scaling law answers the question: If I scale up, what happens to performance?

For LLMs, this becomes:

more parameters + more data + more compute ⇒ predictable reduction in loss

key word

Researchers can run many relatively small experiments, fit a scaling law, and then extrapolate to estimate what may happen at much larger training scales.

The researchers first train models at smaller computer budgets, then estimate a relationship such as:

$L = f(N, D, C)$ and use it to predict the loss of much larger models.

Scaling law does not mean "bigger is always better"

Suppose your total computer budget is fixed: $C \approx KND$.

There is usually an optimal allocation:

$N_{opt}(C)$, $D_{opt}(C)$

This is called compute-optimal scaling

Mo Tu We Th Fr Sa Su

Date / /

Resoure Accounting:

Question: What's the best model one can train give fixed resources (compute, memory)?

In other words: maximize (computational) efficiency.

Question: How long would it take to train a 7oB parameter model on 15T tokens on 1024 H100s?

~~Total~~ total_flops = 6 × 7oe9 × 15e12 C ≈ 6ND

h100_flop_per_sec = 1979e12/2

mfu = 0.5 Model FLOPs Utilization

4ND+2ND: backward pass; forward pass

flops_per_day = h100_flop_per_sec × 6o × 6o × 24

days = total_flops/flops_per_day = 143.9266.

Question 2: What's the largest model you can train on 8 H100s using AdamW (naively)?

h100_bytes = 8oe9

bytes_per_parameter = 2+2+(4+4) # parameters, gradient, optimizer state.

模型参数本身，你如使用BF16：16bits=2bytes
训练时，每个参数对应一个梯度，若梯度也为BF16，则为2bytes.

Mo Tu We Th Fr Sa Su

Memo No. _____

Date / /

num_parameters = (H100_bytes x 8) / bytes_per_parameter

Tensors Basics

Tensors are the basic building block for storing everything.

张量包括向量、矩阵，甚至可以表示任意阶的数组。
array

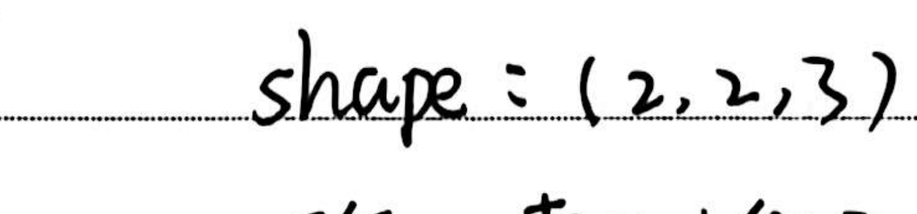
数组的维数与形状：

```
a = [                          形状不确定：
    [
        [1,2,3],               最外层有2个元素 → 2
        [4,5,6],               数组
    ],                         各个元素里有2个元素 → 2
    [                          每个数组里又有3个数字 → 3
        [7,8,9],
        [10,11,12]             shape = (2,2,3)
    ]                          由于每3个索引才能定位一个数字，故是3维数组
]
                               a[0][0][1] = 2
```

控制数字大致数量级

float:

float32:



代表AdamW的两个optimizer states，AdamW对每个参数维护两个

重要变量：m_t, v_t（通常用FP32表示）

一阶矩　二阶矩

Mo Tu We Th Fr Sa Su

Memo No. _____

Date / /

Memory is determined by the: (1) the number of values (2) data type of each value

```
x= torch.zeros(4,8)
assert x.dtype ==torch.float32 # Default type.
assert x.numel == 4x8
assert x.element_size() == 4
assert get_memory_usage(x) == 4x8x4
```

float16、bfloat16、fp8、fp4 这里就不一一介绍了，关键要根据数据范围与精度合理选择

By default, tensors are stored in CPU 内存 memory. However, in order to take advantage of the massive parallelism of GPUs, we need to move them to GPU 显存 memory.

Computing with tensors:

einops:

einstein operations: is a python library design for tensor rearrangement, reshaping, reduction, and repetition.

Mo | Tu | We | Th | Fr | Sa | Su

Memo No. _____

Date / /

motivition: As tensors become higher-dimensional, operation as reshape, permute, transpose, and view become difficult to read and easy to get wrong.

core idea: describe tensor transformation using ~~used~~ named dimension, rather than relying only on numerical dimension indices. 灵感来源于爱因斯坦求和约定

Eg: Easy to mess up the dimensions (what's -2, -1?)...

Traditional PyTorch code:

```
x = torch.ones(2, 2, 3)      # batch seq hidden
y = torch.ones(2, 2, 3)      # batch seq hidden
z = x @ y.transpose(-2, -1)  # batch seq seq
```

y.transpose(-2,-1).shape = (2,3,2)

x @ y.transpose(-2,-1) 就是在两个 batch 中分别做矩阵乘法：2x3 @ 3x2 → 2x2

∴ Z.shape = 2x2x2

Mo Tu We Th Fr Sa Su

Memo No. _____

Date / /

einops_einsum:

x = torch.ones(3,4) # seq hidden

y = torch.ones(4,3) # hidden seq

old way:

z = x @ y # seq1 seq2

new way: (einops)

z = einsum(x, y, "seq1 hidden, hidden seq2 -> seq1 seq2")

类似于爱因斯坦求和中消去了 对应矩阵维度

$$x = \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} \qquad y = (y_1\ y_2\ y_3)$$

$$z = seq_1\ seq_2 = \begin{pmatrix} x_1 \cdot y_1 & x_1 \cdot y_2 & x_1 \cdot y_3 \\ x_2 \cdot y_1 & \cdots & \cdots \\ \cdots & \cdots & \cdots \end{pmatrix}$$

eg:

z = einsum(x, y, "... seq1 hidden, ... seq2 hidden -> ... seq1 s

einops_reduce():

reduce: 按指定的维度模式，把某些维度通过sum、mean、max、min等操作消除掉。

Mo Tu We Th Fr Sa Su

Memo No. _____

Date / /

eg: x = torch.ones(2,3,4) # batch seq hidden

old way:

y = x.sum(dim=-1)

#new way:

y = reduce(x, "... hidden -> ...", "sum")

=

4	4	4
4	4	4

einops_rearrange():

eg:

x = torch.ones(3,8) #seq total_hidden

flattened

...where total_hidden is a flatted representation of

heads * hidden 1

break up `total_hidden` into two dimensions (`head` and 'hidden1'

x = rearrange(x, "... (heads hidden1) -> ... heads hidden1", heads=2)

x.shape = (3,2,4)

w = torch.ones(4,4) # hidden1 hidden2

perform the transformation by w

heads

x = einsum(x, w, "... heads hidden1, hidden1 hidden2 -> ... heads hidden2")

#combine heads and hidden2 together

x = rearrange(x, "... heads hidden2 -> ... (heads hidden2)")

Mo Tu We Th Fr Sa Su

Date / /

Tensor_Operations_FLOPs():

A floating-point operations (FLOPs) is a basic operation like addition (x+y) or multiplication (xy).

FLOPs: floating-point operations : 表示计算量

FLOP/s: floating-point operations per second (also written as FLOPS) : 衡量硬件性能

linear model:

Suppose we have a linear model.

- We have n points
- Each point is d-dimensional
- The linear model maps each d-dimensional vector to a k outputs

x=torch.ones(B, D, devices=cuda_if_available())

w=torch.ones(D, k, ...)

y=x @ w

actual_FLOPs = 2*B*D*k

计算对于 y_{ij} 时需进行 D次乘法, D-1次加法, 此处近似为2D

Mo Tu We Th Fr Sa Su

Memo No. _____

Date / /

Interpretation:

· B is the number of data points

· (DK) is the number of parameters

· FLOPs for forward pass is 2 (#tokens) (#parameters)

It turns out this generalizes to Transformers (to a first-order approximation)

How do FLOPs calculations translate to wall-clock time (seconds)?

Let us time it!

↗ matrix multiplication

actual_time = time_matmul (x, w)

actual_flop_per_sec = actual_total_flops / actual_time

```
def time_matmul (a: torch.Tensor , b: torch.Tensor) -> float:
    if torch.cuda.is_available( ):
        torch.cuda.synchronize( )
    def run( ):
        a @ b
    if torch.cuda.is_available( ):
        torch.cuda.synchronize( )
```

Mo Tu We Th Fr Sa Su

Date /

```
num_trials = 5

total_time = timeit.timeit(run, number=num_trials)

return total_time/num_trials
```

Each GPU has a specification sheet that reports the peak performance.

Note that the FLOP/s depends heavily on the data type!

```
promised_flop_per_sec = get_promised_flop_per_sec(x.type).
```

Model FLOPs utilization (MFU)

MFU = (actual FLOP/s)/(promised FLOP/s) [ignore communicat overhead]

算术强度/运算强度

arithmetic_intensity():



How to compute a thing:

1. Sends inputs from memory to accelerat
2. Perform computation
3. Send outputs from accelerator to memory

Mo Tu We Th Fr Sa Su

Memo No. _____

Date / /

How long does it take? It depends on two things:

1. Accelerator speed (FLOP/s)

2. Memory bandwidth (byte/s)

assert h100_flop_per_sec == 1979e12 /2 # Half wichout sparsity

assert h100_bytes_per_sec == 3.35e12

↗ rectified linear unit 线性整流单元

def arithemetic_intensity_relu(): · f(x)=max(0,x)

n = 1024 * 1024

x = torch.ones(n, dtype=torch.bfloat16, device=cuda_if_avaliable())

y = torch.relu(x)

bytes = (2*n) + (2*n) # Read x, Write y

flops = n # n comparisons

需移动的字节数 ↗

communication_time = bytes / h100_bytes_per_sec = 1.252e-6

computation_time = flops / h100_flops_per_sec = 1.060e-9

通常通信与计算是可以同时进行的

∴ total_time = max(communication_time, computation_time)

Mo Tu We Th Fr Sa Su

Memo No. _____

Date / /

what is bottleneck?

· Memory-bound : communication_time > computation_time

· Compute-bound: ... < ...

In this case, ReLU is memory-bound

Alternative way to see this:

Accelerator intensity VS Arithmetic intensity

↓ ↓

How much work can the accelerator do per byte transferred

How much actual work is performed per byte for this workload

计算任务/负载(指交给计算机系统去执行的具体计算工作量)

=

h100_flop_per_sec / h100_bytes-per-sec

=

flops / bytes

此时:

Memory-bound : arithmetic_intensity < accelerator_intensity

Compute-bound : ... > ...

Mo Tu We Th Fr Sa Su

Memo No. _____

Date / /

assert arithmetic_intensity < h100_accelerator_intensity

注：python 中，assert 是一个关键字/语句（而不是内置函数），用于在代码中声明某个条件必为真。若条件为假，程序会立即抛出 AssertionError 异常并中断执行，常用于内部调试与测试，

In general, we'll find ourselves memory bound, so higher arithmetic intensity is good.（避免大量时间耗费在转移数据上）

```
def arithmetic_intensity_dot_product():
    n=1024
    x=torch.ones(n, dtype=torch.bfloat16, device=cuda-if-avaliable)
    w=torch.ones(
    )
    y=x@w
    bytes=(2*n)+(2*n)+2
    flops=2*n-1
    arithmetic_intensity=flops/bytes  # ≈1/2
    # Memory-bound!
```

Mo Tu We Th Fr Sa Su

Memo No. _____

Date / /

同理可算得：

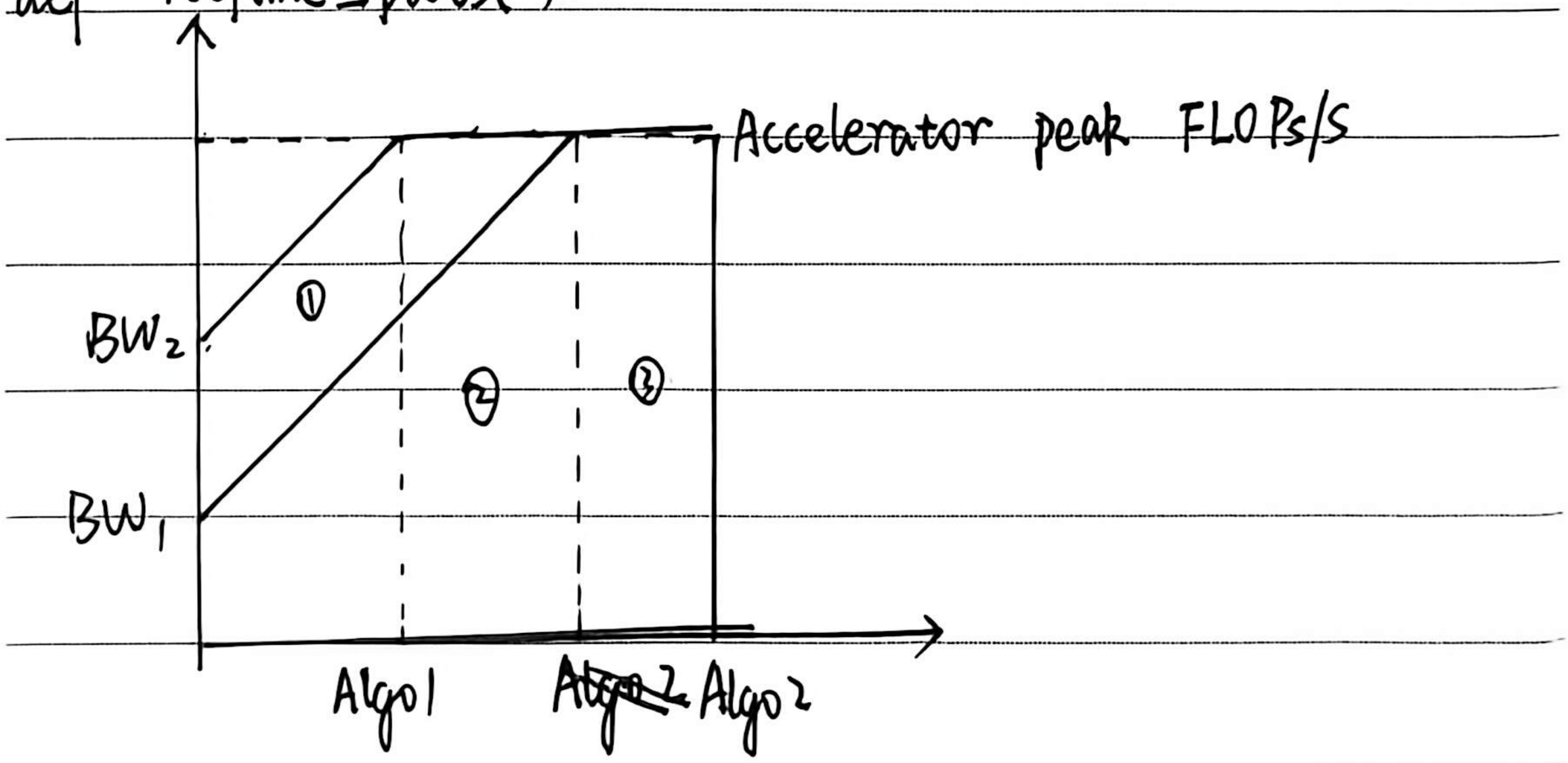
arithmetic_intensity_matrix_vector_product ≈ 1 ⇒ Memory-bound

arithmetic_intensity_~~matrix~~ matmul ≈ n/3 ⇒ Compute-bound (n较大时)

解释了为啥常使用大矩阵乘矩阵，因为n较小时，如果你没有达到加速器强度，实际上不会让速度更快。

Training Transformers involves big matrix multiplications. Matrix-vector product is what happens during inference, which is why inference is memory-bound.

屋顶线图



①: Bandwidth Bound at BW1 & BW2

② Bandwidth Bound at BW1, Compute Bound at BW2

③

Mo Tu We Th Fr Sa Su

Memo No. _____

Date / /

```
def deep_linear_network():
# Consider a deep network with L Layers and D-dimensional
inputs, activations and outputs.


# Define the network
D=8
L=3
model = DeepNetwork(dim=D, num_Layers=L).to(cuda_if_avaliable())


num_parameters = get_num_parameters(model)
assert num_parameters == (D*D)*L


# Run the model on a batch of data
B=4      # Batch size
x=torch.randn(B,D, device=cuda_if_avaliable())
y=model(x)
```